

ICLR 2026



Optimal Sparsity of Mixture-of-Experts Language Models

for Reasoning Tasks

Taishi Nakamura, Satoki Ishikawa, Masaki Kawamura, Takumi Okamoto

Daisuke Nohara, Jun Suzuki, Rio Yokota

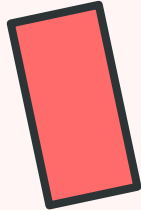
a

Institute of Science Tokyo

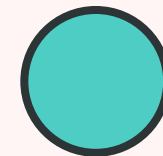
NII LLMC

Tohoku University

RIKEN



Research Overview



01

Introduction & Motivation

The scaling law challenge and research objectives

02

Background

MoE architecture, scaling laws, and post-training methods

03

Experimental Setup

Architecture, hyperparameters, data, and evaluation

04

Key Finding 1: Active FLOPs

Why active compute matters more than training loss

05

Key Finding 2: TPP

Tokens per parameter and task-specific scaling

06

Post-Training & TTC Impact

GRPO and test-time compute analysis

07

Coding Task Validation

Extending findings to code generation

08

Discussion & Conclusions

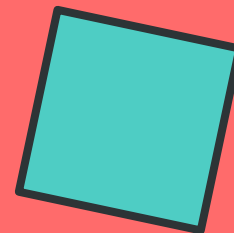
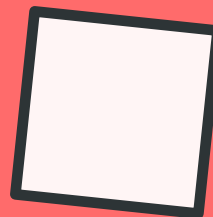
Principles, recommendations, and future work



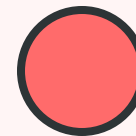
CHAPTER 01

Introduction & Research Motivation

Understanding the sparsity-reasoning paradox in MoE language models



The Scaling Law Challenge



Evolution of Scaling Laws

Kaplan et al. (2020)

Established power-law relationships between model performance, size, and compute across **seven orders of magnitude** for dense Transformers

Hoffmann et al. (2022)

Demonstrated that scaling curves can be inverted to choose **compute-optimal** combinations of parameters and tokens for fixed budgets



The MoE Revolution

Mixture-of-Experts (MoE) has become the de facto standard for state-of-the-art systems:

■ Gemini 2.5 Pro (2025)

■ DeepSeek-V3 (2025)

■ Qwen3 (2025)



The Critical Gap

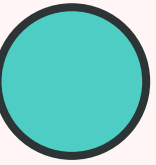
Scaling law coefficients are **NOT universal**. Highly expressive models trained under different optimizers or architectures often follow the same loss trajectory but diverge substantially on downstream reasoning benchmarks.



The Core Problem

MoE introduces a new **sparsity dimension** that current dense-model frontiers overlook. The classical picture of compute-optimal scaling is incomplete for MoE architectures.

Research Questions & Objectives



Core Research Question

How does MoE sparsity influence **memorization skills** versus **reasoning skills** in large language models?



Memorization Skills

TriviaQA

Reading comprehension & knowledge retrieval

HellaSwag

Commonsense reasoning & sentence completion

Parameter-hungry tasks



Reasoning Skills

GSM8K

Grade school math word problems

GSM-Plus

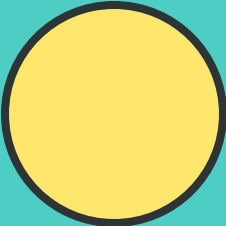
Robust math problem solving benchmark

Data-hungry tasks



Investigation Scope

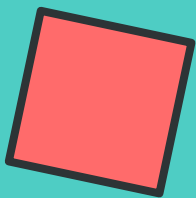
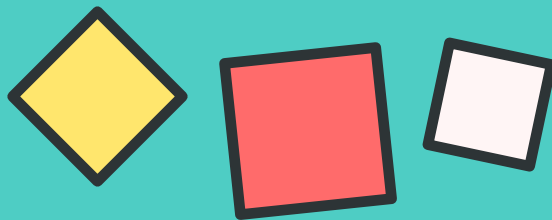
Train MoE families varying **total parameters**, **active parameters**, and **top-k routing** under fixed compute budgets to disentangle pre-training loss from downstream accuracy.



CHAPTER 02

Background & Related Work

Foundations of MoE architecture and scaling laws



Mixture-of-Experts Architecture



MoE Layer Mechanics

Token Representation

$$x \in \mathbb{R}^{d_h}$$

Router Scores

$$s = x^T W_{\text{router}} \in \mathbb{R}^n$$

Top-k Selection

Select indices K of k largest components



Gating Function

$$g(x)_i = \exp(s_i) / \sum_{j \in K} \exp(s_j) \text{ if } i \in K, \text{ else } 0$$



Layer Output

$$y = \sum_{i=1}^n g(x)_i \text{FFN}(x)_i$$

Weighted sum of chosen experts



Auxiliary Losses

Load-Balancing Loss (LLB)

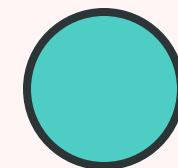
Prevents expert collapse

Router-Z Loss (LRz)

Penalizes large router logits

$$L = L_{\text{CE}} + \alpha L_{\text{LB}} + \beta L_{\text{RZ}}$$

Scaling Laws Evolution



1

Kaplan et al. (2020)

Power-law relationships:

$$L \propto N^{-\alpha}, L \propto D^{-\beta}$$

Across 7 orders of magnitude for dense Transformers

2

Hoffmann et al. (2022)

Compute-optimal scaling:

$$N_{\text{opt}} \propto C^{0.50}, D_{\text{opt}} \propto C^{0.50}$$

Joint optimization of parameters and tokens

3

MoE Scaling Research

- Clark et al. (2022): Unified scaling laws
- Frantar et al. (2024): Sparsity-aware exponents
- Abnar et al. (2025): Parameters vs FLOPs frontier

! The Task Loss Challenge

Next-token prediction loss does NOT necessarily align with downstream task performance

Key Insight:

Models with identical training loss can exhibit markedly different reasoning performance

≡ Skills-Based Scaling

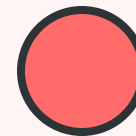
Knowledge Tasks

"Capacity-hungry" - benefit from larger model sizes

Code/Reasoning Tasks

"Data-hungry" - benefit from increased training data

Post-Training & Test-Time Compute



Reinforcement Learning Post-Training

PPO (Proximal Policy Optimization)

Backbone of RLHF pipelines from GPT alignment to GPT-4

GRPO (Group Relative Policy Optimization)

Replaces value function with group-relative advantage estimator. Powers DeepSeek-R1 for mathematical reasoning



GRPO Benefits

- ✓ Improved memory efficiency
- ✓ Stabilized updates
- ✓ State-of-the-art math reasoning



Test-Time Compute (TTC)

Accuracy gains without updating model parameters:

1

Longer Chains of Thought

Running extended reasoning sequences

2

Larger Candidate Pools

Sampling multiple answers

3

Self-Consistency

Majority-vote aggregation



Research Gap

How do these methods interact with MoE sparsity? Can they recover degraded reasoning from suboptimal sparsity choices?



CHAPTER 03

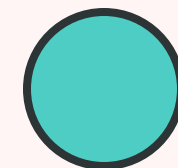
Experimental Methodology



Systematic investigation of MoE scaling behavior



Model Architecture & Hyperparameters



Base Architecture

Mixtral-style Transformer with:

- RMSNorm
- SwiGLU activations
- Rotary positional embeddings
- Dropless token-choice top-k routing

L = 16 layers



Swept Hyperparameters

Model Width (d)
{512, 1024, 2048}

Experts (E)
{8, 16, 32, 64, 128, 256}

Top-k
{2, 4, 8, 16}



Training Config

Optimizer: AdamW

Peak LR: 4×10^{-4}

Warmup: 2k steps linear

Decay: Cosine

Weight Decay: 0.1

FFN hidden dim: 2d



Auxiliary Loss Weights

Load-Balancing

$\alpha = 10^{-2}$

Router-Z

$\beta = 10^{-3}$

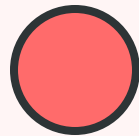


Hyperparameter Study

To isolate optimization effects:

- Fixed E = 16, k = 2
- Widths: $d \in \{512, 1024\}$
- Varied: LM-head initialization
- Varied: Peak learning rate
- Varied: AdamW ϵ

Training Data & Evaluation Protocol



Pre-training Corpus

125B Tokens

High-quality web text

43B

Mathematics corpora

32B

STEM literature

49B

Code

1B



Task Loss Computation

Cross-entropy computed only over answer tokens by concatenating prompt with ground-truth answer



Evaluation Benchmarks

GSM8K

Grade school math word problems

4-shot

GSM-Plus

Robust math problem solving

5-shot CoT

TriviaQA

Reading comprehension

4-shot

HellaSwag

Commonsense reasoning

4-shot



Measurements

Pre-training
Loss

Task
Loss

Downstream
Accuracy



CHAPTER 04

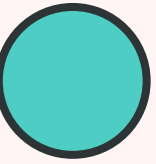
Key Finding 1

Active FLOPs Matter

Downstream reasoning depends on active compute, not just training loss



The Divergence Phenomenon



Key Observation

Training and validation loss decrease monotonically with total parameters, but task loss on GSM8K worsens beyond a threshold



Memorization Tasks

TriviaQA

Task loss follows scaling trend - improves monotonically with parameter increase

HellaSwag

Consistent improvement as training loss decreases

✓ Expected scaling behavior



Reasoning Tasks

GSM8K

Exhibits U-shaped behavior - task loss initially decreases then increases

GSM-Plus

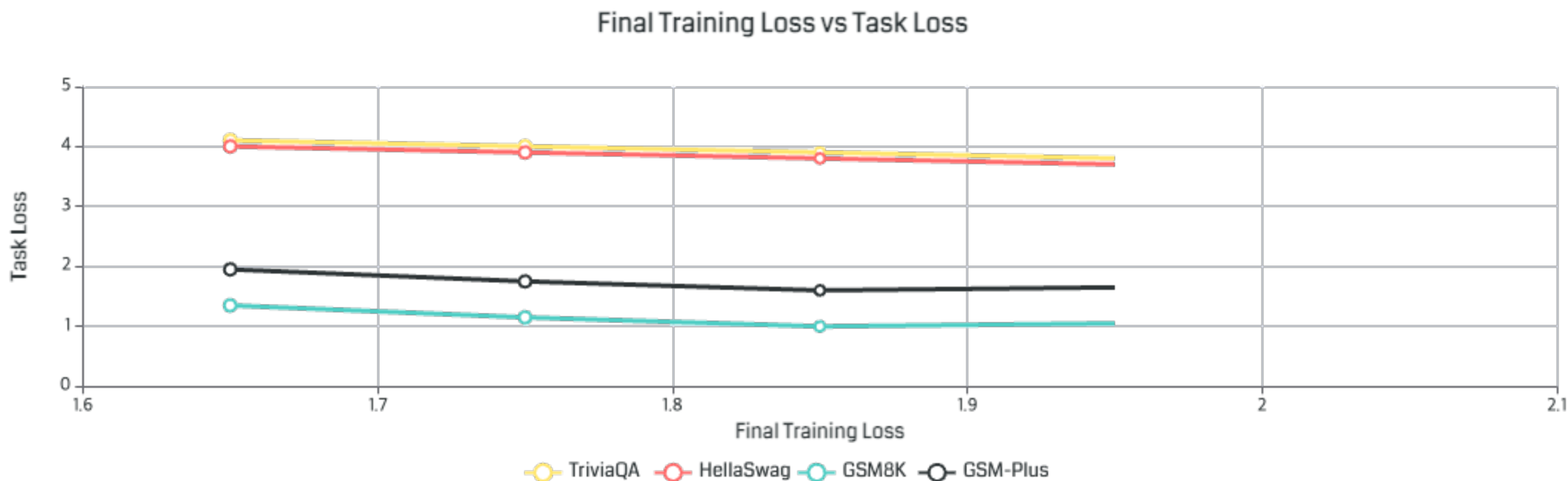
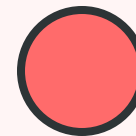
Same non-monotonic trend as GSM8K

✗ Unexpected divergence



Experimental Condition: Top-k and layer width held constant while increasing expert count (and thus total parameters)

Training Loss vs Task Loss Analysis



TriviaQA
Monotonic ↓

HellaSwag
Monotonic ↓

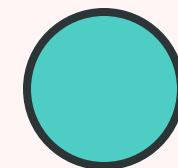
GSM8K
U-shaped

GSM-Plus
U-shaped



Key Insight: The threshold where task loss starts increasing moves **lower** as active parameter count increases. Models with more active parameters achieve lower optimal task loss.

Accuracy Trends & Sparsity Dependence



Memorization: Monotonic Improvement

t

For TriviaQA and HellaSwag:

- Accuracy improves steadily as training loss decreases
- No performance degradation with more parameters
- Consistent with classical scaling laws



Reasoning: Non-Monotonic Behavior

r

For GSM8K:

- Further reductions in training loss do NOT always improve accuracy
- Can actually **degrade performance**
- Over-optimizing pre-training loss harms reasoning



Sparsity Dependence Analysis

s

Memorization Skills

Error rate tracks training loss and is largely insensitive to sparsity

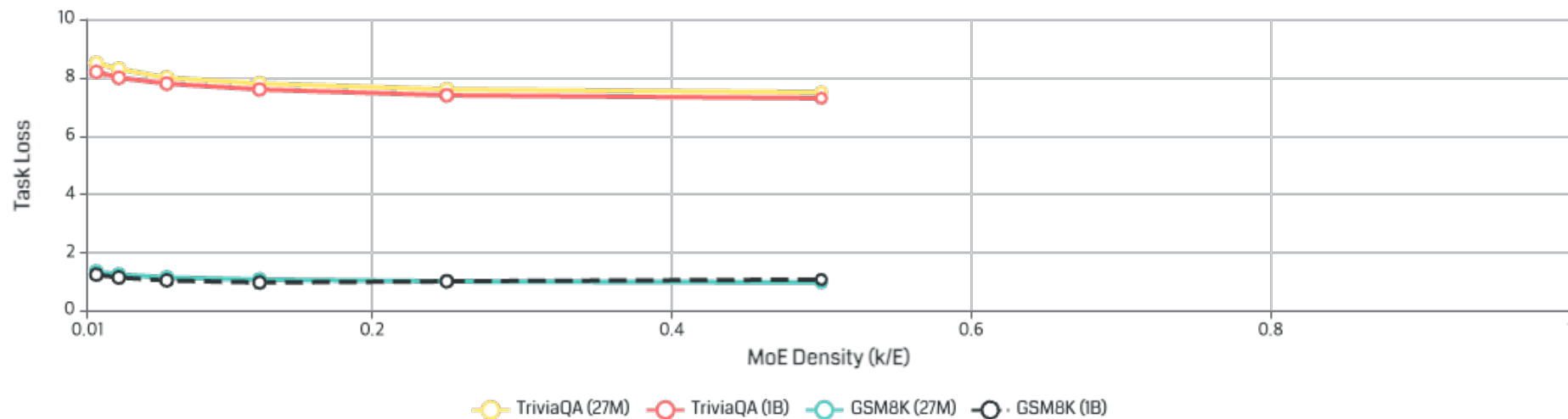
Reasoning Skills

Exhibit strong dependence of error rate on sparsity

Iso-FLOP Analysis: Optimal Sparsity



Task Loss vs MoE Density (Iso-FLOP)



Memorization Tasks

Pattern

Higher sparsity (lower density) consistently improves performance

Applies To

All active parameter budgets

Reasoning Tasks

Low FLOPs

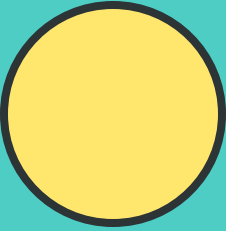
Increasing sparsity reduces loss

High FLOPs

Denser models outperform sparser ones



Key Finding: Reasoning skills admit a **compute-dependent optimum**, rather than monotonically favoring either sparsity or density

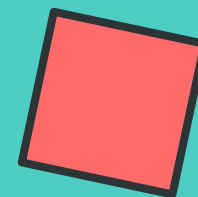
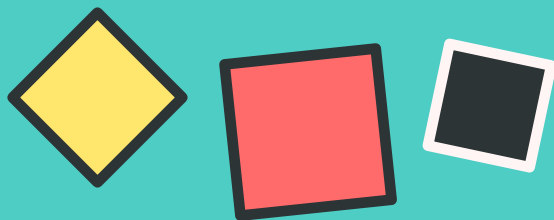


CHAPTER 05

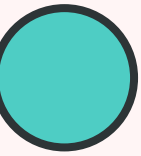
Key Finding 2

Tokens Per Parameter (TPP)

TPP governs task-specific scaling behavior



TPP and Task Performance



Chinchilla Scaling Law

Hoffmann et al. (2022)

~20 tokens per parameter

Optimal ratio for dense models under fixed compute budgets



Memorization Skills

Behavior

Performance improves monotonically as TPP decreases

Characterization

"Parameter-hungry" - benefit from more parameters

Lower TPP → Better Performance



Reasoning Skills

Behavior

Non-monotonic trend - accuracy peaks near TPP ≈ 20

Characterization

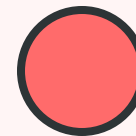
"Data-hungry" - benefit from more training data

Optimal TPP ≈ 20

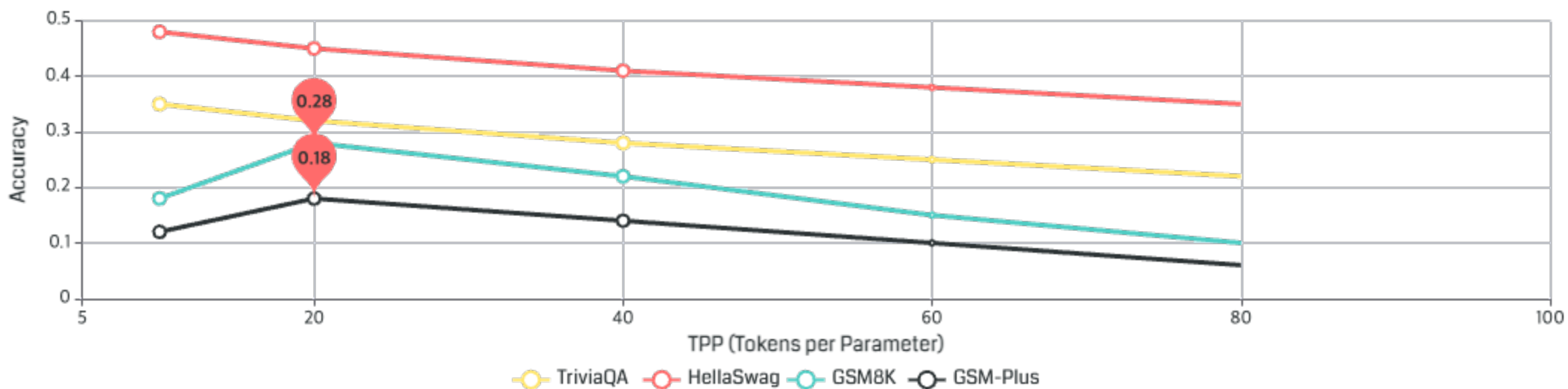


Critical Finding: Even at fixed TPP, models with **larger top-k** consistently outperform those with smaller top-k on reasoning task

The Data-Parameter Balance



Accuracy vs Tokens Per Parameter (TPP)



Low TPP Regime

Too Many Parameters

Reasoning degrades due to data starvation

Optimal TPP

TPP $\approx$ 20

Peak reasoning performance

High TPP Regime

Too Few Parameters

Reasoning suffers from under-parameterization



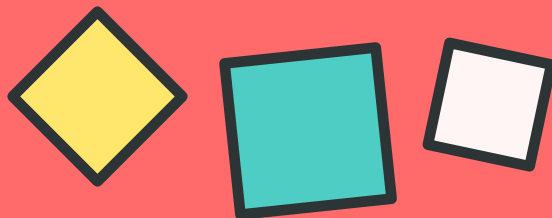
MoE-Specific Insight: Reasoning depends on both **Total TPP** and the balance between **total** and **active** parameters. Compute-optimal scaling must explicitly consider both.



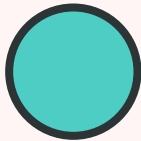
CHAPTER 06

Post-Training & Test-Time Compute Impact

Can GRPO and TTC recover degraded reasoning?



GRPO Post-Training Results



GRPO Experimental Setup

Batch Size	1024
Training Duration	15 epochs
Total Steps	10^5
Actor LR	5×10^{-6}
Temperature	1.0
KL-penalty	10^{-3}
Samples per Prompt	5



GRPO Results

Finding: GRPO improves overall performance across all model configurations

- ✓ Consistent accuracy gains
- ✓ Scales with model size
- ✓ Benefits both sparse and dense models

BUT: Critical Limitation

GRPO does **NOT eliminate** the inverted U-shaped relationship between training loss and task accuracy



Key Insight



Performance Gap Persists

The gap from increased sparsity remains unchanged after GRPO



Trade-off Unchanged

Memorization vs reasoning trade-off is not resolved by post-training



Implication

Finding optimal sparsity during pre-training is crucial

Test-Time Compute Analysis



TTC Setup

Method

Self-Consistency (SC) decoding

Continuations

27 independent per problem

Temperature

0.6

Nucleus Sampling

top-p = 0.95

Samples

128 per problem



TTC Results

Finding: TTC yields performance improvements that scale with model size

- ✓ Higher accuracy across all models
- ✓ Benefits both sparse and dense configurations
- ✓ Consistent with prior TTC research

BUT: Trade-off Persists

The loss-accuracy trade-off on GSM8K remains unchanged



Comprehensive Findings



Neither TTC nor GRPO

mitigate the GSM8K performance drop from increased total parameters



Both methods improve

overall performance but do NOT eliminate the inverted U-shaped relationship

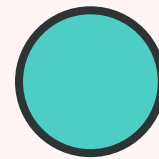


Pre-training sparsity



Critical implication

Key Implications



Synthesis of Findings

1

Pre-training Dominance

Sparsity choices during pre-training are the **primary determinant** of reasoning ability

2

Complementary but Limited

Post-training & TTC are **complementary** but cannot compensate for suboptimal sparsity

3

Crucial Decision Point

Finding optimal MoE sparsity during pre-training is **essential** for reasoning models



What Works

✓ Careful Sparsity Selection

Balance active FLOPs with TPP during pre-training

✓ Task-Aware Design

Different sparsity for memorization vs reasoning tasks

✓ Compute-Dependent Optimization

Adjust density based on available FLOPs budget



What Doesn't Work

X Blind Sparsity Increase

More experts always better - FALSE for reasoning

X Post-Training Fixes

GRPO cannot recover from suboptimal sparsity

X Universal TPP

One-size-fits-all TPP ignores task differences



CHAPTER 07

Coding Task Validation

Extending findings to code generation



Code Generation Experiments



Training Setup

Corpus Size	125B Tokens
Stack-Edu Python	95B
DCLM-dedup Web	30B
Same architecture & hyperparameters as main experiments	

Evaluation Benchmarks

HumanEval	0-shot
Pass@1 metric	
HumanEval+	0-shot
Extended test cases	
MBPP	3-shot
Pass@1 metric	
MBPP+	3-shot
Extended test cases	

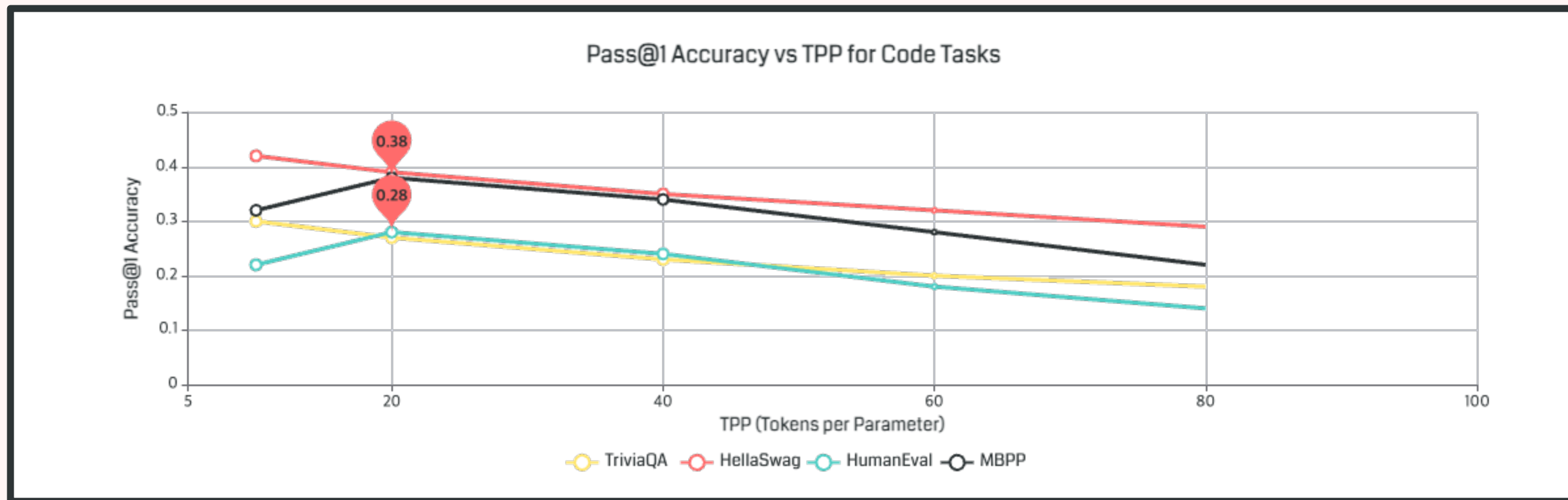
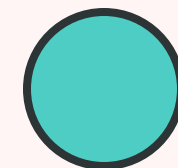
Key Finding: Results Mirror Math Reasoning

Low Active Parameters
Higher sparsity (lower density) improves performance
Consistent with memorization tasks

High Active Parameters
Denser models outperform sparser counterparts
Shift toward density at scale

Conclusion: Code generation exhibits the same sparsity-performance trade-offs as mathematical reasoning

Code Task TPP Analysis



TriviaQA

Monotonic ↑
with more params

HellaSwag

Monotonic ↑
with more params

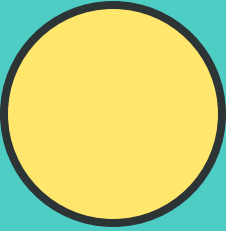
HumanEval

Non-Monotonic
optimal TPP exists

MBPP

Non-Monotonic
optimal TPP exists

✓ **Validation:** Coding tasks follow the same TPP patterns as mathematical reasoning, confirming the **generality** of our findings across procedural reasoning tasks

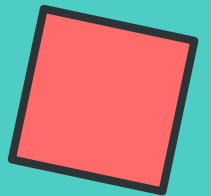
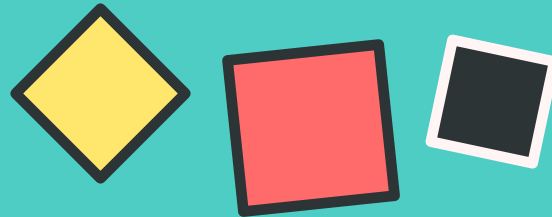


CHAPTER 08

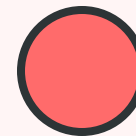


Discussion & Conclusions

Synthesizing insights for MoE scaling practice



Two Core Principles



1 Active FLOPs Matter

Core Insight

Downstream reasoning quality is determined by active FLOPs at both training and test time, not pre-training loss alone

Key Finding

Larger top-k consistently outperforms smaller ones even when pre-training loss is matched

Implication

Active compute is a critical dimension for reasoning performance

2 TPP Matters

Core Insight

The tokens-per-parameter ratio governs task-specific scaling behavior

Memorization

"Parameter-hungry" - improves with lower TPP (more parameters)

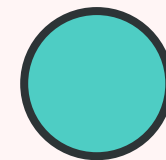
Reasoning

"Data-hungry" - peaks near $TPP \approx 20$, degrades when TPP is too low or too high



Together, these principles refine current scaling practice: Sparsity and more experts improve memorization under fixed budgets, but reasoning requires balancing active FLOPs with TPP. In high-compute regimes, optimal density depends jointly on compute and data.

Practical Recommendations



For Memorization Tasks

✓ Increase Sparsity

More experts and higher sparsity consistently improve performance

✓ Lower TPP

Use more parameters relative to training tokens

✓ Maximize Capacity

Parameter-hungry tasks benefit from larger model sizes



For Reasoning Tasks

✓ Balance Active FLOPs

Larger top-k values improve reasoning even at matched loss

✓ Target TPP ≈ 20

Optimal ratio for data-hungry reasoning tasks

✓ Compute-Dependent Density

At high FLOPs, denser configurations may be optimal



High-Compute Regime Guidance

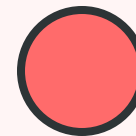
Limited Data Available

Denser MoE layers preserve reasoning performance

Abundant Data Available

Greater sparsity remains advantageous for efficiency

Limitations & Future Work



Study Limitations

Token Budget

125B tokens (Chinchilla-optimal for dense models of comparable activated size)

Fixed Token Budget

All experiments held tokens fixed; larger corpora could shift optimal sparsity

Architecture Focus

Mixtral architecture without shared experts or QK-norm

Design Space

Vast MoE design space; systematic sweep focused on width, expert count, top-k



Future Directions

Larger Training Corpora

Investigate how 1T+ tokens shift optimal sparsity for reasoning

Exhaustive Architecture Search

Explore shared experts, QK-norm, alternative routing schemes

Larger-Scale Validation

Test findings on models with 10B+ active parameters

Extended Training Studies

Analyze how training duration affects sparsity-reasoning trade-offs



Important Note

With larger training corpora, one could train larger models at higher tokens-per-parameter, potentially shifting optimal sparsity toward sparser configurations even for reasoning skills. We leave this as a direction for future work.



Open Science Commitment

Thank You for Your Attention

All resources are open-source and available at:

github.com/rioyokotalab/optimal-sparsity



Code



Checkpoints



Logs

